数学知识

数学知识

- 算数基本定理
- 质数
  - AcWing 866. 试除法判定质数
  - 867. 分解质因数
  - 868. 筛质数
    - 埃式筛法：
      - cpp
      - python
    - 线性筛法：
- 约数
  - 869. 试除法求约数
  - 870. 约数个数
  - 871. 约数求和
  - 872. 最大公约数
- 欧拉函数
  - 873. 欧拉函数
  - 874. 筛法求欧拉函数
- 欧拉定理
  - 欧拉定理和费马丁柳
- 快速幂
  - 875. 快速幂
  - 876. 快速幂求逆元
- 拓展欧几里得算法
- 容斥原理
  - AcWing 890. 能被整除的数
- 博弈论
  - 公平组合游戏（NIM游戏）
  - SG函数
  - AcWing 891. Nim游戏
  - AcWing 892. 台阶-Nim游戏
  - AcWing 893. 集合-Nim游戏
  - AcWing 894. 拆分-Nim游戏

算数基本定理

任何大于 1 的整数是**质数**或**独一无二的质数乘积**（不理次序）。

任何一个大于1的自然数n都可以表示为素数的乘积形式：
$[n = p_1^{e_1} \cdot p_2^{e_2} \cdot \ldots \cdot p_k^{e_k}]$。其中，$(p_1, p_2, \ldots, p_k)$是素数，$(e_1, e_2, \ldots, e_k)$是大于等于1的整数，并且这种分解方式是唯一的。

质数

在大于1的整数中，如果只包含1和本身这两个约数，就被称为质数，或者叫素数。

AcWing 866. 试除法判定质数

给定 n 个正整数 a_i，判定每个数是否是质数。

输入格式

第一行包含整数 n。

接下来 n 行，每行包含一个正整数 a_i。

输出格式

共 n 行，其中第 i 行输出第 i 个正整数 a_i 是否为质数，是则输出 `Yes`，否则输出 `No`。

质数的判定——试除法 O(sqrt(n))

由于约数是成对出现的，所以n的最大约数只能是$\sqrt{n}$

```
#include <iostream>
#include <algorithm>

const int N = 110;
int a[N];

using namespace std;

bool is_prime (int n)
{
    if (n < 2)  return false;
    for  (int i  = 2; i <= n/i; i ++)
         {
            if (n % i == 0)
                 return false;
         }
    return true;
}


int main()
{
    int n;
    cin >> n;
    for (int i = 0; i < n ; i++)    cin >> a[i];

     for (int i = 0; i < n ; i++)
        {
        if(is_prime (a[i]))
            cout << "Yes" << endl;
```

```
        else
            cout << "No" << endl;
        }

        return 0;

}
```

867. 分解质因数

给定 n 个正整数 a_i，将每个数分解质因数，并按照质因数从小到大的顺序输出每个质因数的底数和指数。

输入格式

第一行包含整数 n。

接下来 n 行，每行包含一个正整数 a_i。

输出格式

对于每个正整数 a_i，按照从小到大的顺序输出其分解质因数后，每个质因数的底数和指数，每个底数和指数占一行。

每个正整数的质因数全部输出完毕后，输出一个空行。

数据范围

$1 \le n \le 100$,
$2 \le a_i \le 2 \times 10^9$

分解质因数　O(sqrt(n))

- 质因数的**底数**指的是质数的基数，比如在分解12=2×2×3的式子中，2和3就是底数。 而**指数**指的是底数出现的次数,底数2的指数为2，底数3的指数为1。
- 由算数基本定理，我们从小到大枚举所有质数，求出底数后再求质数，即可分解成功

```
#include <iostream>
#include <algorithm>

const int N = 110;
int a[N];


using namespace std;

void divide (int x)
{
    for (int i = 2; i <= x/i; i ++)
        if (x % i == 0)
        {
            int s = 0;
            while (x % i == 0)
            {
                x /= i;
```

```
                s ++;
            }
            cout << i << ' ' << s << endl;
        }
        if (x > 1)
            cout << x << ' ' << 1 << endl;
        puts ("");
}




int main()
{
    int n;
    cin >> n;
    for (int i = 0; i < n ; i++)    cin >> a[i];

     for (int i = 0; i < n ; i++)
        {
        divide (a[i]);
        }

        return 0;

}
```

868. 筛质数

给定一个正整数 n，请你求出 $1 \sim n$ 中质数的个数。

输入格式

共一行，包含整数 n。

输出格式

共一行，包含一个整数，表示 $1 \sim n$ 中质数的个数。

数据范围

$1 \leq n \leq 10^6$

质数定理:1~n中有n/lnn个质数。

埃式筛法:

cpp

```
#include <iostream>
using namespace std;
```

```
const int N = 1e6 + 10;

int primes[N], cnt;
bool st[N];

void get_primes(int n)
{
    for(int i = 2; i <= n; i++)
    {
        if(!st[i])  //若为质数
        {
            primes[cnt++] = i;
                for(int j = i + i; j <=n; j+=i)
                    st[j] = true;
        }


    }
}

int main() {
    int n;
    cin >> n;
    get_primes(n);
    cout << cnt << endl;
    return 0;


}
```

python

```
n = int(input())
cnt = 0
st = [True] * (n+10)

for i in range(2,n+1):
    if st[i]:   #是质数
        cnt+=1
        #z.append(i)
        for i in range(i+i,n+1,i):
            st[i]  = False

print(cnt)
```

线性筛法：

线性筛法：n只会被最小质因子筛掉(适合范围较大)

```
#include <iostream>
using namespace std;
const int N = 1e6 + 10;
```

```

int primes[N], cnt;
bool st[N];
void get_primes(int n) {
    for (int i = 2; i <= n; i++) {
        if (!st[i]) { // 若当前数字i是素数
            primes[cnt++] = i; // 将i存入primes数组
            for (int j = 0; primes[j] <= n / i; j++) {
                st[primes[j] * i] = true; // 标记i的倍数为非素数
                if (i % primes[j] == 0) break; // 保证每个合数只被其最小的素因数筛掉
            }
        }
    }
}


int main() {
  int n;
  cin >> n;
  get_primes(n);
  cout << cnt << endl;
  return 0;
}
```

约数

869. 试除法求约数

给定 n 个正整数 a_i，对于每个整数 a_i，请你按照从小到大的顺序输出它的所有约数。

输入格式

第一行包含整数 n。

接下来 n 行，每行包含一个整数 a_i。

输出格式

输出共 n 行，其中第 i 行输出第 i 个整数 a_i 的所有约数。

试除法求所有约束,时间复杂度$\sqrt{n}$

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

void solve(int n) {
  vector<int> vec;
  for (int i = 1; i <= n / i; i++) {
    if (n % i == 0)
```

```
        {
          vec.push_back(i);
          if (i != n / i)
              vec.push_back(n / i);
        }

    }
    sort(vec.begin(),vec.end());
      for(auto i : vec)
      {
          cout << i << " ";
      }
      cout << endl;
}

int main() {
    int n;
    cin >> n;
    while(n --)
    {
      int x;
      cin >> x;
      solve(x);
    }

}
```

870. 约数个数

$[约数个数 = (c_1 + 1) \cdot (c_2 + 1) \cdots (c_k + 1)]$

给定 n 个正整数 a_i，请你输出这些数的乘积的约数个数，答案对 10^9+7 取模。

输入格式

第一行包含整数 n。

接下来 n 行，每行包含一个整数 a_i。

输出格式

输出一个整数，表示所给正整数的乘积的约数个数，答案需对 10^9+7 取模。

数据范围

$1 \le n \le 100$,
$1 \le a_i \le 2 \times 10^9$

输入样例：

```
3
2
6
8
```

输出样例：

```
12
```

```cpp
#include <iostream>
#include <unordered_map>

using namespace std;

long long mod = 1e9 + 7;

int main() {
  int n;
  cin >> n;
  unordered_map<int, int> cnt;
  while (n--) {
    int x;
    cin >> x;
    for (int i = 2; i <= x / i; i++) {
      while (x % i == 0) {
        x /= i;
        cnt[i]++;
      }
    }
    if (x > 1)       //n也是一个质因子
      cnt[x]++;
  }
  long long ans = 1;
  for (auto c : cnt) {
       //res = (x1+1)(x2+1)(x3+1)…(xk+1)
    ans = ans * (c.second + 1) % mod;
  }
  cout << ans << endl;
```

```
}

```

871. 约数求和

给定 n 个正整数 a_i，请你输出这些数的乘积的约数之和，答案对 10^9+7 取模。

输入格式

第一行包含整数 n。

接下来 n 行，每行包含一个整数 a_i。

输出格式

输出一个整数，表示所给正整数的乘积的约数之和，答案需对 10^9+7 取模。

数据范围

$1 \le n \le 100$,
$1 \le a_i \le 2 \times 10^9$

输入样例：

```
3
2
6
8
```

输出样例：

```
252
```

$[\text{约数之和} = (\sum_{i=0}^{c_1} p_1^i) \cdot (\sum_{i=0}^{c_2} p_2^i) \cdots (\sum_{i=0}^{c_k} p_k^i)]$

```
#include <iostream>
#include <unordered_map>

using namespace std;
#define LL long long
long long mod = 1e9 + 7;

int main() {
  int n;
  cin >> n;
  unordered_map<int, int> cnt;
  while (n--) {
    int x;
    cin >> x;
    for (int i = 2; i <= x / i; i++) {
      while (x % i == 0) {
        x /= i;
        cnt[i]++;
      }
    }
    if (x > 1)      //n也是一个质因子
```

```
        cnt[x]++;
    }
    long long res = 1;

      for (auto p : cnt)
      {
          LL a = p.first, b = p.second;
          LL t = 1;
          while (b -- ) t = (t * a + 1) % mod;
          //t = 1   t = p + 1   t = p^2 + p + 1
          res = res * t % mod;
      }

      cout << res << endl;



    cout << ans << endl;
}

```

872. 最大公约数

给定 n 对正整数 a_i, b_i，请你求出每对数的最大公约数。

输入格式

第一行包含整数 n。

接下来 n 行，每行包含一个整数对 a_i, b_i。

输出格式

输出共 n 行，每行输出一个整数对的最大公约数。

数据范围

$1 \le n \le 10^5$,
$1 \le a_i, b_i \le 2 \times 10^9$

输入样例：

```
2
3 6
4 6
```

输出样例：

```
3
2
```

欧几里得算法（辗转相除法）：

(a， b) = (a % b， b) = （b, a %b）

```
#include <iostream>
using namespace std;

```

```

int gcd(int a, int b)
{
    return b == 0? a : gcd(b, a % b);
}


int main()
{
    int n, a, b;
    cin >> n;
    while(n--)
    {
        cin >> a >> b;
        cout << gcd(a,b) << endl;
    }
}
```

欧拉函数

873. 欧拉函数

给定 n 个正整数 a_i，请你求出每个数的欧拉函数。

欧拉函数的定义

> $1 \sim N$ 中与 N 互质的数的个数被称为欧拉函数，记为 $\phi(N)$。
> 若在算数基本定理中，$N = p_1^{a_1} p_2^{a_2} \dots p_m^{a_m}$，则：
> $\phi(N) = N \times \frac{p_1-1}{p_1} \times \frac{p_2-1}{p_2} \times \dots \times \frac{p_m-1}{p_m}$

输入格式

第一行包含整数 n。

接下来 n 行，每行包含一个正整数 a_i。

输出格式

输出共 n 行，每行输出一个正整数 a_i 的欧拉函数。

如果 n 的质因数分解为：
$[n = p_1^{k_1} \times p_2^{k_2} \times \cdots \times p_m^{k_m}]$

那么，欧拉函数可以表示为：
$[\varphi(n) = n\left(1-\frac{1}{p_1}\right)\left(1-\frac{1}{p_2}\right)\cdots\left(1-\frac{1}{p_m}\right)]$

```
#include <iostream>

using namespace std;
const int  N = 100010;

typedef long long ll;
```

```cpp

void solve()
{
    int n;
    cin >> n;
    int res = n;
    for(int i = 2; i <= n / i; i++)
    {
        if(n % i == 0)
        {

            // res*=(p - 1) / p
            res = res / i * (i - 1);
            while(n % i == 0)   //对n约分
            {
                n /= i;
            }
        }
    }
    // 如果有剩余，则剩余是个质因子
    if(n > 1)
    {
        res = res / n * (n - 1);
    }
    cout <<res << endl;
}
int main()
{
    int n;
    cin >> n;
    while(n--)
    {
        solve();
    }
}


```

874. 筛法求欧拉函数

给定一个正整数 n，求 $1 \sim n$ 中每个数的欧拉函数之和。

输入格式

共一行，包含一个整数 n。

输出格式

共一行，包含一个整数，表示 $1 \sim n$ 中每个数的欧拉函数之和。

数据范围

$1 \le n \le 10^6$

输入样例：

```
6
```

输出样例：

```
12
```

[AcWing 874. 筛法求欧拉函数 - AcWing]

[AcWing 874. 筛法求欧拉函数 - AcWing]

phi[primes[j] * i]分为两种情况：
① i % primes[j] == 0时：primes[j]是i的最小质因子，也是primes[j] * i的最小质因子，因此1 - 1 / primes[j]这一项在phi[i]中计算过了，只需将基数N
N
修正为primes[j]倍，最终结果为phi[i] * primes[j]。
② i % primes[j] != 0：primes[j]不是i的质因子，只是primes[j] * i的最小质因子，因此不仅需要将基数N
N
修正为primes[j]倍，还需要补上1 - 1 / primes[j]这一项，因此最终结果phi[i] * (primes[j] - 1)。

```cpp
#include <iostream>

using namespace std;

typedef long long LL;
const int N = 1000010;

int primes[N], cnt;
int phi[N];
bool st[N];

void get_eulers(int n) {
  phi[1] = 1;
  for (int i = 2; i <= n; i++) {
    if (!st[i]) {
      primes[cnt++] = i;
      phi[i] = i - 1; // 若为质数，有i - 1个数与其互质
    }
```

```
    for (int j = 0; primes[j] <= n / i; j++) {
      st[primes[j] * i] = true;
      if (i % primes[j] == 0) //最小质因子
      {
        phi[primes[j] * i] = phi[i] * primes[j];
        /*
  当p[j]是i的一个约数时,i的质因数与p[j]*i的质因数完全相同。
  i      = (p1^a1)*(p2^a2)*(p3^a3)*...(p[j]^ap[j])...(pk^ak)
  i*p[j] = (p1^a1)*(p2^a2)*(p3^a3)*...(p[j]^(ap[j]+1))...(pk^ak)
  由欧拉函数的定义可知:
  Φ(i)      =  i    * (1-1/p1)*(1-1/p2)*(1-1/p3)*...(1-1/p[j])*...(1-1/pk)
  Φ(i*p[j]) = i*p[j]*(1-1/p1)*(1-1/p2)*(1-1/p3)*...(1-1/p[j])*...(1-1/pk)
  */
        break;
      }
      /*
当i%p[j]!=0时，p[j]不是i的约数，i与i*p[j]的质因子相差一个p[j]
i      = (p1^a1)*(p2^a2)*(p3^a3)*...(pk^ak)
i*p[j] = (p1^a1)*(p2^a2)*(p3^a3)*...(pk^ak)*(p[j]^ap[j])
所以由欧拉函数的定义可知:
Φ(i)      =  i    *(1-1/p1)*(1-1/p2)*(1-1/p3)*...(1-1/pk)
Φ(i*p[j]) = i*p[j]*(1-1/p1)*(1-1/p2)*(1-1/p3)*...(1-1/pk)(1-1/p[j])
*/
      phi[primes[j] * i] = phi[i] * (primes[j] - 1);
    }
  }
}

int main() {
  int n;
  cin >> n;

  get_eulers(n);

  LL res = 0;
  for (int i = 1; i <= n; i++)
    res += phi[i];
  printf("%lld\n", res);

  return 0;
}

```

欧拉定理

欧拉定理是数论中的一个重要定理，它描述了在模 m 的整数环中，与 m 互质的整数 a 和 m 的欧拉函数 $\varphi(m)$ 之间的关系。以下是欧拉定理的表述：
如果 m 是一个正整数，a 是一个与 m 互质的正整数，那么 $a^{\varphi(m)} \equiv 1 \pmod{m}$。
其中，$\varphi(m)$ 表示小于或等于 m 的与 m 互质的正整数的个数，这个函数被称为欧拉函数。
以下是欧拉定理的Markdown格式表示：

```
# 欧拉定理
欧拉定理是数论中的一个基本定理，其内容如下：
如果 \( m \) 是一个正整数，\( a \) 是一个与 \( m \) 互质的正整数，那么：
\[ a^{\varphi(m)} \equiv 1 \pmod{m} \]
这里，\(\varphi(m)\) 是欧拉函数，表示小于或等于 \( m \) 的与 \( m \) 互质的正整数的个数。
```

当你将上述Markdown文本放入支持Markdown的编辑器中时，它将被渲染为格式化的文本，如下所示：

欧拉定理和费马丁柳

欧拉定理：
如果 m 是一个正整数，a 是一个与 m 互质的正整数

那么：$a^{\varphi(m)} \equiv 1 \pmod{m}$

费马小定理:
如果 p 是一个素数（有p-1个数与其互质），而 a 是一个不被 p 整除的整数，那么：
$a^{p-1} \equiv 1 \pmod{p}$

快速幂

875. 快速幂

给定 n 组 a_i, b_i, p_i，对于每组数据，求出 $a_i^{b_i} \bmod p_i$ 的值。

输入格式

第一行包含整数 n。

接下来 n 行，每行包含三个整数 a_i, b_i, p_i。

输出格式

对于每组数据，输出一个结果，表示 $a_i^{b_i} \bmod p_i$ 的值。

每个结果占一行。

```
#include <iostream>

using namespace std;

typedef long long LL;



int qmi(int a, int k, int p)
{
    LL res = 1;
    while(k)
    {
        if(k & 1) res = (LL)res * a % p;        //注意转型，因为我们传入的类型为int
        k = k >> 1;
        //更新a,a依次为a^{2^0},a^{2^1},a^{2^2},....,a^{2^logb}
```

```
        a = (LL)a * a % p;
    }
    return res;
}



int main()
{
    int n;
    cin >> n;
    while(n--)
    {
        int a, k , p;
        cin >> a >> k >> p;
        cout << qmi(a, k, p) << endl;
    }
}
```

876. 快速幂求逆元

给定 n 组 a_i, p_i，其中 p_i 是质数，求 a_i 模 p_i 的乘法逆元，若逆元不存在则输出 `impossible`。

注意：请返回在 $0 \sim p-1$ 之间的逆元。

乘法逆元的定义

> 若整数 b，m 互质，并且对于任意的整数 a，如果满足 $b|a$，则存在一个整数 x，使得 $\frac{a}{b} \equiv a \times x \pmod{m}$，则称 x 为 b 的模 m 乘法逆元，记为 $b^{-1} \pmod{m}$。
>
> b 存在乘法逆元的充要条件是 b 与模数 m 互质。当模数 m 为质数时，b^{m-2} 即为 b 的乘法逆元。

输入格式

第一行包含整数 n。

接下来 n 行，每行包含一个数组 a_i, p_i，数据保证 p_i 是质数。

输出格式

输出共 n 行，每组数据输出一个结果，每个结果占一行。

若 a_i 模 p_i 的乘法逆元存在，则输出一个整数，表示逆元，否则输出 `impossible`。

费马定理+快速幂

[AcWing 876. 快速幂求逆元 - AcWing]

a / b ≡ a * x (mod n)
两边同乘b可得 a ≡ a * b * x (mod n)
即 1 ≡ b * x (mod n)
同 b * x ≡ 1 (mod n)
由费马小定理可知，当n为质数时
b ^ (n - 1) ≡ 1 (mod n)
拆一个b出来可得 b * b ^ (n - 2) ≡ 1 (mod n)
故当n为质数时，b的乘法逆元 x = b ^ (n - 2)

```
#include <iostream>

using namespace std;

typedef long long LL;

//a^k%p
int qmi(int a, int k, int p)
{
    LL res  = 1;
    while(k)
    {
        if(k & 1) res = (LL)res * a % p;
        a = (LL)a * a % p;
        k >>= 1;
    }
    return res;
}

int main()
{
    int n;
    cin >> n;
    while(n--)
    {
        int a, p;
        cin >> a >> p;
        int res = qmi(a, p - 2, p);
        if(a % p)  cout << res << endl; //互质存在逆元
        else cout <<"impossible"<< endl;
    }
}
```

拓展欧几里得算法

容斥原理

并集

$$|A_1 \cup A_2 \cup \cdots \cup A_n| = \sum_{i=1}^{n} |A_i| - \sum_{1 \le i < j \le n} |A_i \cap A_j| + \sum_{1 \le i < j < k \le n} |A_i \cap A_j \cap A_k| - \cdots + (-1)^{n+1} |A_1 \cap A_2 \cap \cdots \cap A_n|$$

交集

$$|A_1 \cap A_2 \cap \cdots \cap A_n| = \sum_{i=1}^{n} |A_i| - \sum_{1 \le i < j \le n} |A_i \cup A_j| + \sum_{1 \le i < j < k \le n} |A_i \cup A_j \cup A_k| - \cdots + (-1)^{n+1} |A_1 \cup A_2 \cup \cdots \cup A_n|$$

共有

$$2^n$$

种情况

即——对于集合个数与符号的关系，奇数符号为正，偶数符号为负

AcWing 890. 能被整除的数

890. 能被整除的数

题目 提交记录 讨论 题解 视频讲解

给定一个整数 n 和 m 个不同的质数 $p_1, p_2, \ldots, p_m$。

请你求出 $1 \sim n$ 中能被 $p_1, p_2, \ldots, p_m$ 中的至少一个数整除的整数有多少个。

输入格式

第一行包含整数 n 和 m。

第二行包含 m 个质数。

输出格式

输出一个整数，表示满足条件的整数的个数。

数据范围

$1 \le m \le 16$,
$1 \le n, p_i \le 10^9$

输入样例：

```
10 2
2 3
```

输出样例：

```
7
```

```cpp
#include <iostream>
using namespace std;

typedef long long LL;

const int N = 20;

int p[N];

int main() {
    int n, m;
    cin >> n >> m;

    // 输入 m 个质数
    for (int i = 0; i < m; i++) {
        cin >> p[i];
    }

    int res = 0;

    // 枚举所有可能的质数组合 ，所有组合数和为2^n
```

```cpp
    for (int i = 1; i < 1 << m; i++) {
        int product = 1, count = 0;
        //质数乘积，count表示包含几个集合

        // 检查当前子集是否包含第 j 个质数
        for (int j = 0; j < m; j++) {
            if (i >> j & 1) {
                // 如果乘积超过 n，跳过当前子集
                if ((LL)product * p[j] > n) {
                    product = -1;
                    break;
                }
                product *= p[j];    //并集大小
                count++;    //集合个数++
            }
        }

        // 根据容斥原理更新结果
        if (product != -1) {
            if (count&1) {      //奇数个集合，符号为正
                res += n / product;
            } else {            //偶数个集合，符号为负
                res -= n / product;
            }
        }
    }

    cout << res << endl;

    return 0;
}
```

博弈论

公平组合游戏（NIM游戏）

公平组合游戏（Impartial Game）的定义如下：

- 游戏有两个人参与，二者轮流做出决策，双方均知道游戏的完整信息；
- 任意一个游戏者在某一确定状态可以作出的决策集合只与当前的状态有关，而与游戏者无关；

- 游戏中的同一个状态不可能多次抵达，游戏以玩家无法行动为结束，且游戏一定会在有限步后以非平局结束。

- **异或和为0**：当前玩家处于**必败态**，无论怎么操作，对手总能迫使你回到必败态。
- **异或和非0**：当前玩家处于**必胜态**，存在一种操作使对手进入必败态。

SG函数

1. **SG值为0**：如果 SG(G)=0SG(*G*)=0，则当前状态是必败态（P-position），即当前玩家无法必胜。
2. **SG值不为0**：如果 SG(G)≠0SG(*G*)\=0，则当前状态是必胜态（N-position），即当前玩家可以通过策略将游戏转移到必败态。
3. S**G函数理论**多个独立局面的SG值，等于这些局面SG值的异或和。

```
memset(f,-1,sizeof(f));
f[M],s[N];//s存储的是可供选择的集合,f存储的是所有可能出现过的情况的sg值


int sg(int x) {
    if (f[x] != -1) return f[x];  // 记忆化：已计算过则直接返回
    set<int> S;                   // 存储当前状态x的所有后继状态的SG值，用set去重
    // 遍历所有合法操作（s数组为允许的取法集合，如每次可取1、2、3个石子）
    for (int i = 0; i < m; i++) {
        int sum = s[i];           // 当前允许取的石子数（如s[i]=1,2,3）
        if (x >= sum)
            S.insert(sg(x - sum)); // 递归计算后继状态x-sum的SG值
    }
    // 计算Mex：找最小的未出现在S中的自然数
    for (int i = 0; ; i++)
        if (!S.count(i))
            return f[x] = i;      // 保存并返回当前状态的SG值
}
```

AcWing 891. Nim游戏

给定 n 堆石子，两位玩家轮流操作，每次操作可以从任意一堆石子中拿走任意数量的石子（可以拿完，但不能不拿），最后无法进行操作的人视为失败。

问如果两人都采用最优策略，先手是否必胜。

输入格式

第一行包含整数 n。

第二行包含 n 个数字，其中第 i 个数字表示第 i 堆石子的数量。

输出格式

如果先手方必胜，则输出 `Yes`。

否则，输出 `No`。

数据范围

$1 \le n \le 10^5$,
$1 \le$ 每堆石子数 $\le 10^9$

输入样例：

```
2
2 3
```

输出样例：

```
Yes
```

```cpp
int main() {
    ios::sync_with_stdio(0);
    cin.tie(0);
    cin >> n;
    int res = 0;
    for(int i = 0; i < n; i++)
    {
        cin >> arr[i];
    }
    for(int i = 0; i < n; i++)
    {
        res ^= arr[i];
    }
    if(res == 0)
    {
        cout << "No" << endl;
    }
    else {
        cout << "Yes" << endl;
    }
}
```

AcWing 892. 台阶-Nim游戏

现在，有一个 n 级台阶的楼梯，每级台阶上都有若干个石子，其中第 i 级台阶上有 a_i 个石子($i \geq 1$)。

两位玩家轮流操作，每次操作可以从任意一级台阶上拿若干个石子放到下一级台阶中（不能不拿）。

已经拿到地面上的石子不能再拿，最后无法进行操作的人视为失败。

问如果两人都采用最优策略，先手是否必胜。

输入格式

第一行包含整数 n。

第二行包含 n 个整数，其中第 i 个整数表示第 i 级台阶上的石子数 a_i。

输出格式

如果先手方必胜，则输出 `Yes`。

否则，输出 `No`。

数据范围

$1 \leq n \leq 10^5$,
$1 \leq a_i \leq 10^9$

输入样例：

```
3
2 1 3
```

输出样例：

```
Yes
```

分为奇数阶和偶数阶

- 对于偶数阶，若存在某偶数级台阶石子数影响胜负，则玩家可将其移动到下一级奇数级台阶，改变奇数级异或和。但对手可立即将这些石子继续传递到后续台阶，最终消除其对奇数级异或和的影响。
- 对于奇数阶，则是由这些奇数台阶石子堆组成的经典Nim游戏问题。所有石子都可以通过偶数次操作（还是先手）转化到一阶

```
int main() {
    ios::sync_with_stdio(0);
    cin.tie(0);
    cin >> n;
    for(int i = 0; i < n; i++)
    {
        cin >> arr[i];
    }
    int res = 0;
    for(int i = 0; i < n; i+=2)
    {
        res ^= arr[i];
    }
```

```
    if(res == 0)
    {
        cout << "No" << endl;
    }
    else {
        cout << "Yes" << endl;
    }
}

```

AcWing 893. 集合-Nim游戏

给定 n 堆石子以及一个由 k 个不同正整数构成的数字集合 S。

现在有两位玩家轮流操作，每次操作可以从任意一堆石子中拿取石子，每次拿取的石子数量必须包含于集合 S，最后无法进行操作的人视为失败。

问如果两人都采用最优策略，先手是否必胜。

输入格式

第一行包含整数 k，表示数字集合 S 中数字的个数。

第二行包含 k 个整数，其中第 i 个整数表示数字集合 S 中的第 i 个数 s_i。

第三行包含整数 n。

第四行包含 n 个整数，其中第 i 个整数表示第 i 堆石子的数量 h_i。

输出格式

如果先手方必胜，则输出 `Yes`。

否则，输出 `No`。

数据范围

$1 \le n, k \le 100,$
$1 \le s_i, h_i \le 10000$

输入样例：

```
2
2 5
3
2 4 7
```

SG的核心作用是将复杂的博弈问题转化为等效的 Nim游戏

```
#include <cstring>
#include <iostream>
#include <utility>
#include <set>
using namespace std;
typedef long long LL;
const int N = 1e6 +10;
int arr[N];
```

```
int S[N];

typedef pair<int, int> PA;
typedef pair<PA, int> PA2;
int k,n;


int  SG(int x) {
    if(S[x]!= -1)           //记忆化搜索
        return S[x];
    set<int> status;
    for(int i = 0; i < k; i++)
    {
        int sum = arr[i];
        if (x >= sum)
            status.insert(SG(x - sum));
    }
    for (int i = 0; ; i++)
        if (!status.count(i))
            return S[x] = i;
}

int main() {
    ios::sync_with_stdio(0);
    cin.tie(0);
    cin >> k ;
    int res = 0;
    memset(S, -1, sizeof S);
    for(int i = 0;i < k; i++)
    {
        cin >> arr[i];
    }
    cin >> n;
    for(int i = 0; i < n; i++)
    {
        int tmp;
        cin >> tmp;

        res ^= SG(tmp);
    }
    if(res)
        cout << "Yes" << endl;
    else
        cout << "No" << endl;

}
```

AcWing 894. 拆分-Nim游戏

给定 n 堆石子，两位玩家轮流操作，每次操作可以取走其中的一堆石子，然后放入两堆**规模更小**的石子（新堆规模可以为 0，且两个新堆的石子总数可以大于取走的那堆石子数），最后无法进行操作的人视为失败。

问如果两人都采用最优策略，先手是否必胜。

输入格式

第一行包含整数 n。

第二行包含 n 个整数，其中第 i 个整数表示第 i 堆石子的数量 a_i。

输出格式

如果先手方必胜，则输出 `Yes`。

否则，输出 `No`。

数据范围

$1 \le n, a_i \le 100$

输入样例：

```
2
2 3
```

输出样例：

```
Yes
```

- 任意一个较小堆的拆分情况会被完全包含在较大堆中（sg嵌套sg）
- 采用递归，求出x时的sg()函数，
- 使用全局unordered_set维护sg(x)的函数值，进行剪枝优化（第一次调用最耗时）

```
#include <cstring>
#include <iostream>
#include <utility>
#include <unordered_set>
using namespace std;
typedef long long LL;
const int N = 1e5 +10;
int arr[N];

typedef pair<int, int> PA;
typedef pair<PA, int> PA2;
int n,m;
unordered_set<int> st;

int ss[N];

int sg(int x) {
    if(ss[x] != -1) return ss[x];

    for(int i = 0; i < x; i++)
```

```
        for(int j = 0; j<= i; j++)
    {
        st.insert(sg(i)^sg(j));
    }

    for(int i = 0; ;i++)
        if(!st.count(i))
            return ss[x] = i;
}

int main() {
    ios::sync_with_stdio(0);
    cin.tie(0);
    cin >> n;
    int res = 0;
    memset(ss, -1, sizeof ss);
    while(n--) {
        int tmp = 0;
        cin >> tmp;
        res ^= sg(tmp);
    }
    if(res)
        cout << "Yes" <<endl;
    else
        cout << "No" << endl;

}

```